

# DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

GRAPHS AND BREADTH FIRST SEARCH

# GRAPH REPRESENTATIONS

# VERTICES AND EDGES

A graph is a pair of sets. Everything else today is either a statement about those two sets or a statement about how you store them.

$$G = (V, E)$$

$V$  is the vertex set. Write  $n$  for  $|V|$ .

$E$  is the edge set. Write  $m$  for  $|E|$ .

An undirected edge is an unordered pair  $\{u, v\}$ . The edge  $\{u, v\}$  and the edge  $\{v, u\}$  are the same element of  $E$ .

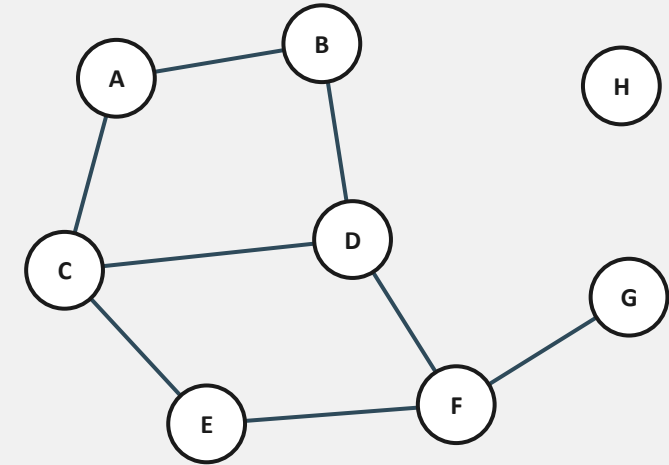
A directed edge is an ordered pair  $(u, v)$  with a tail at  $u$  and a head at  $v$ . Those are two different elements of  $E$ .

**For the graph on the right:**

$$V = \{A, B, C, D, E, F, G, H\}, \quad n = 8$$

$$E = \{ \{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}, \\ \{C, E\}, \{D, F\}, \{E, F\}, \{F, G\} \}, \quad m = 8$$

$H$  is a vertex with no edges. That is legal, and it is going to be useful later when we ask what a traversal actually reaches.



## M IS BOUNDED BY N SQUARED

A simple undirected graph has at most  $n(n-1)/2$  edges, so  $m$  is between 0 and roughly  $n$  squared. Sparse means  $m$  is close to  $n$ . Dense means  $m$  is close to  $n$  squared.

Every bound today is written in both  $n$  and  $m$  because those two ends of the range behave nothing alike.

# THE ADJACENCY MATRIX

An  $n$  by  $n$  array where entry  $[u][v]$  records whether the edge is there. Any question about a single pair of vertices is one array read.

```
adj = [[0] * n for _ in range(n)]

def add_edge(u, v):          # Theta(1)
    adj[u][v] = 1
    adj[v][u] = 1            # undirected only

def has_edge(u, v):          # Theta(1)
    return adj[u][v] == 1

def neighbors(u):             # Theta(n)
    return [v for v in range(n) if adj[u][v]]
```

| WHAT EACH OPERATION TAKES       |            |
|---------------------------------|------------|
| Add or remove an edge           | Theta(1)   |
| Test whether edge (u, v) exists | Theta(1)   |
| List the neighbors of u         | Theta(n)   |
| Visit every edge once           | Theta(n^2) |
| Space                           | Theta(n^2) |

|   | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| D | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| E | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| F | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| G | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| H | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**SPACE DOES NOT DEPEND ON M**

This matrix holds 64 entries and 16 of them are ones. The other 48 are storage reserved for edges that are not there.

Scale that up. A graph with a million vertices and a million edges needs a matrix with a trillion entries. That single sentence is the whole objection to the matrix.

# THE ADJACENCY LIST

One container of neighbors per vertex. The structure stores what is there instead of reserving room for everything that could be there.

```
adj = {u: set() for u in vertices}

def add_edge(u, v):          # Theta(1)
    adj[u].add(v)
    adj[v].add(u)           # undirected only

def has_edge(u, v):          # Theta(1) expected
    return v in adj[u]

def neighbors(u):             # Theta(deg(u))
    return adj[u]
```

## WHAT EACH OPERATION TAKES

|                                 |                                   |
|---------------------------------|-----------------------------------|
| Add or remove an edge           | $\Theta(1)$                       |
| Test whether edge (u, v) exists | $\Theta(1)$                       |
| List the neighbors of u         | $\Theta(\deg(u))$                 |
| Visit every edge once           | $\Theta(n + m)$                   |
| Space                           | <b><math>\Theta(n + m)</math></b> |

|          |              |
|----------|--------------|
| <b>A</b> | B, C         |
| <b>B</b> | A, D         |
| <b>C</b> | A, D, E      |
| <b>D</b> | B, C, F      |
| <b>E</b> | C, F         |
| <b>F</b> | D, E, G      |
| <b>G</b> | F            |
| <b>H</b> | <i>empty</i> |

## WHY SETS AND NOT LISTS

A Python list makes membership testing  $\Theta(\deg(u))$ , so `has_edge` stops being constant. The set is a hash table, and hashing is exactly what buys back the constant time edge test.

## IN NEIGHBORS ARE NOT FREE

In a directed graph this stores out neighbors only. Finding the in neighbors of u means scanning every list, which is  $\Theta(n + m)$ . Store the reversed graph if you need them often.

# CHOOSING A REPRESENTATION

Both are correct. They differ in what they make cheap, and the graph you are handed decides which set of cheap operations you want.

|                     | ADJACENCY MATRIX                        | ADJACENCY LIST                       |
|---------------------|-----------------------------------------|--------------------------------------|
| Shape in Python     | List of n lists of length n             | Dict from vertex to set of neighbors |
| Space               | Theta(n^2), independent of m            | Theta(n + m)                         |
| Test edge (u, v)    | Theta(1), one array read                | Theta(1) expected, one set lookup    |
| List neighbors of u | Theta(n), scan a whole row              | Theta(deg(u))                        |
| Visit every edge    | Theta(n^2)                              | Theta(n + m)                         |
| Constant factor     | Small. Contiguous memory, no pointers   | Larger. Hashing and pointer chasing  |
| Good fit            | Dense graphs, m close to n^2            | Sparse graphs, m close to n          |
| Also good for       | Anything algebraic on the matrix itself | Anything that walks the graph        |

# GRAPH TERMINOLOGY

# DIRECTION AND DEGREE

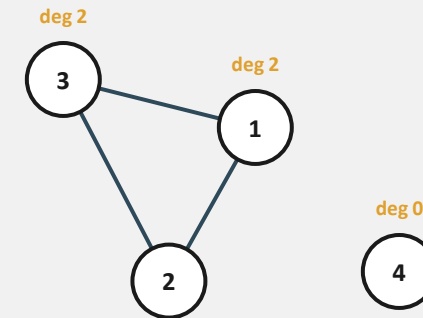
Whether an edge has a direction changes what degree means and changes which questions the graph can answer.

## UNDIRECTED

An edge is an unordered pair  $\{u, v\}$ . Adjacency is symmetric, so if  $u$  is a neighbor of  $v$  then  $v$  is a neighbor of  $u$ .

$\deg(u)$  is the number of edges incident to  $u$ .

Friendship on a social network is undirected. So is a road with two way traffic, and so is an electrical connection.

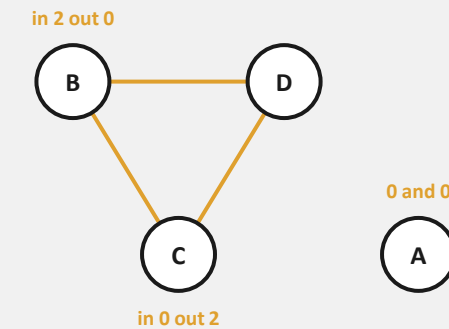


## DIRECTED

An edge is an ordered pair  $(u, v)$  with a tail at  $u$  and a head at  $v$ . Adjacency is no longer symmetric.

$\text{indeg}(u)$  counts edges into  $u$ ,  $\text{outdeg}(u)$  counts edges out of  $u$ , and the two are unrelated.

Following an account is directed. So is a one way street, a task that must finish before another, and an import in a source file.



# WALKS, PATHS, AND CYCLES

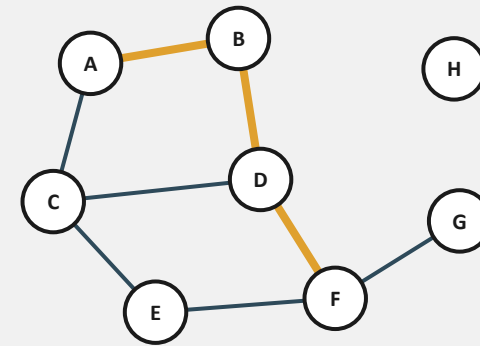
Three words for moving through a graph. They differ only in what you are allowed to repeat, and that difference is what the algorithms turn on.

## WALKS AND PATHS

A walk is a sequence of vertices where every consecutive pair is joined by an edge. Repeats are allowed.

A path is a walk with no repeated vertex.

The length of a walk is the number of edges on it, not the number of vertices. A walk on  $k$  vertices has length  $k$  minus 1.



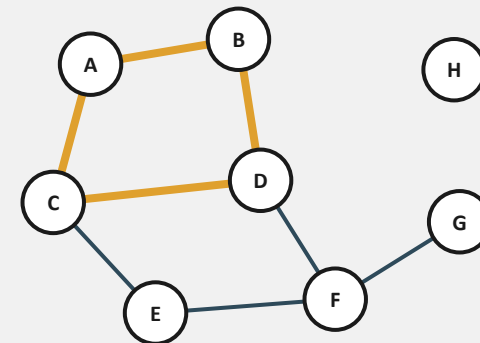
Path A, B, D, F has length 3

## CYCLES

A cycle is a walk that starts and ends at the same vertex and repeats no other vertex.

Its length is again the number of edges, which for a cycle equals the number of distinct vertices on it.

A graph with no cycles at all is called acyclic. Whether a cycle has even or odd length turns out to matter, and the last section of this lecture is about exactly that.



Cycle A, B, D, C, A has length 4

# CONNECTIVITY

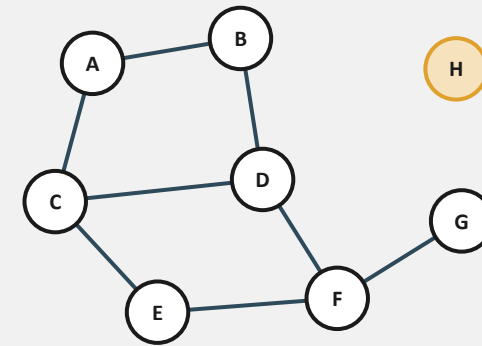
Whether you can get from anywhere to anywhere. The undirected version and the directed version are genuinely different questions.

## UNDIRECTED: CONNECTED, AND COMPONENTS

An undirected graph is connected if there is a path between every pair of vertices.

A connected component is a maximal set of vertices that are all reachable from one another. Every graph splits into components, and no edge crosses between two of them.

Our running graph is not connected. It has two components.

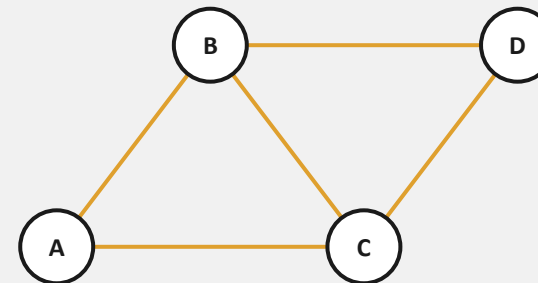


Components: {A, B, C, D, E, F, G} and {H}

## DIRECTED: STRONGLY CONNECTED

A directed graph is strongly connected if for every pair  $u$  and  $v$  there is a directed walk from  $u$  to  $v$  and a directed walk back from  $v$  to  $u$ .

Reachability is now one directional, so  $u$  reaching  $v$  tells you nothing about  $v$  reaching  $u$ . The picture on the right is strongly connected, and deleting the single edge from  $D$  to  $B$  breaks it.



# BUILDING THE GRAPH

Almost nothing arrives as a graph. Turning a problem into one is a modeling step, and it is usually where the difficulty actually is.

## 1. Name the fundamental objects. Those are the vertices.

If you cannot say what one vertex is in a single noun, you are not ready to write code yet.

## 2. Name the relation between two objects that the problem cares about. That is the edge.

One relation, one graph. Two different relations on the same objects give two different graphs.

## 3. Ask whether the relation is symmetric.

Symmetric means undirected. If  $u$  relates to  $v$  but  $v$  may not relate to  $u$ , the graph is directed.

## 4. Ask whether the relation carries a number.

If each edge has a cost attached, the graph is weighted. If every edge is interchangeable, it is unweighted.

### Worked example: fewest connecting flights between two cities.

Vertices are airports. Edges are scheduled flights. Directed, since a flight from San Diego to Chicago is not a flight from Chicago to San Diego. Unweighted, because the question asked for the fewest flights and not the least total time in the air.

### THE SAME SYSTEM GIVES DIFFERENT GRAPHS

Change the question to least total flight time and the same airports and flights become a weighted graph, and breadth first search stops being the right tool. The modeling step encodes the question, not just the data.

### GRIDS ARE GRAPHS YOU NEVER BUILD

A grid of cells is a graph where vertices are cells and edges join cells sharing a side. Nobody builds the adjacency list. You compute the neighbors from the coordinates when you need them. Same algorithms, implicit representation.

### A VERTEX DOES NOT HAVE TO BE A THING

A vertex can be an entire configuration and an edge a legal move between configurations. Puzzle solvers are breadth first search over that graph, and the graph is far too large to write down.

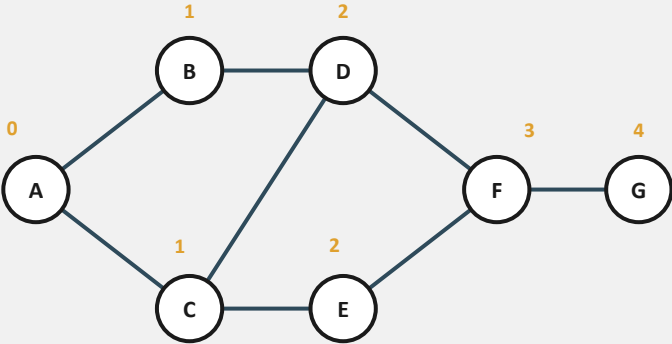
BREADTH FIRST SEARCH

# TRAVERSALS

A traversal visits the vertices of a graph in some order. The order is the entire algorithm, and swapping one data structure changes it completely.

## What you use one for

Decide whether  $v$  is reachable from  $u$ . Find a shortest walk from  $u$  to  $v$ . Count the connected components. Detect a structure such as a cycle, or decide whether the graph can be split in two. Nearly every graph problem is a traversal with bookkeeping bolted on.



Breadth first order from A: A, B, C, D, E, F, G

## TWO ORDERS

|               | HOLDS WORK IN         | VISITS NEXT                  |
|---------------|-----------------------|------------------------------|
| Breadth first | A queue               | The closest unvisited vertex |
| Depth first   | A stack, or recursion | The deepest unvisited vertex |

Those two rows differ in one line of code

### THE CONTAINER IS THE ALGORITHM

Replace the queue with a stack and the visit order changes completely while every other line stays identical. There is nothing else separating the two traversals.

### MARKING IS NOT OPTIONAL

Without a visited set a cycle sends the traversal around forever, and a vertex reachable by many walks gets expanded once per walk.

# BREADTH FIRST SEARCH IN PYTHON

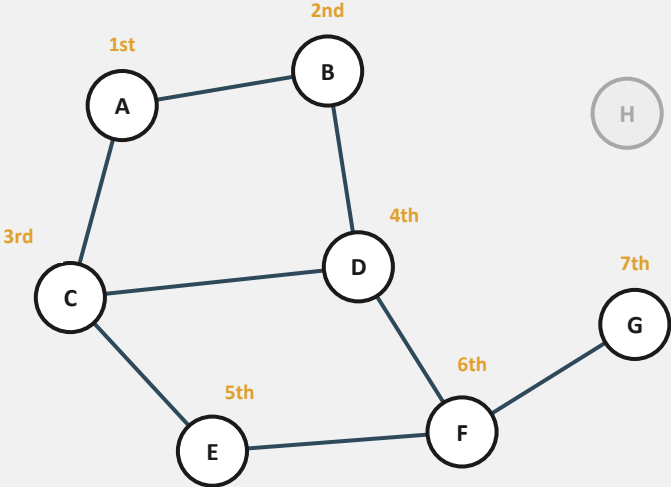
Mark a vertex when you put it in the queue, not when you take it out. That one choice is what keeps any vertex from entering the queue twice.

```
from collections import deque

def bfs(adj, source):
    visited = {source}
    queue = deque([source])

    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in visited:
                visited.add(v)      # mark on enqueue
                queue.append(v)

    return visited
```



| POPPED | QUEUE AFTER THE POP | NEWLY MARKED |
|--------|---------------------|--------------|
| A      | B, C                | B, C         |
| B      | C, D                | D            |
| C      | D, E                | E            |
| D      | E, F                | F            |
| E      | F                   | nothing      |
| F      | G                   | G            |
| G      | empty               | nothing      |

**WHY MARKING ON DEQUEUE IS A BUG**

If two already queued vertices both point at w, w gets appended twice before either copy is popped. The queue can then grow past n entries and the running time proof on the next slide stops working.

**WHAT IT RETURNS**

Exactly the set of vertices reachable from source. H never appears, because nothing reaches it. That gap is the next slide.

# WHY BREADTH FIRST SEARCH IS LINEAR

Read as nested loops the code looks like  $\Theta(n \cdot m)$ . The true bound is  $\Theta(n + m)$ , and closing that gap is what aggregate analysis is for.

## The loose reading.

A while loop that runs up to  $n$  times, containing a for loop that could touch up to  $m$  edges, gives  $O(nm)$ . That bound is true and it is not tight. Do not analyze the loops. Analyze the events.

## How many pops happen in total?

A vertex is appended only inside the branch that first adds it to visited, and a vertex enters visited exactly once. So there are at most  $n$  appends across the entire run, and therefore at most  $n$  pops. The while loop body runs at most  $n$  times, not  $n$  times per anything.

## How much neighbor scanning happens in total?

One pop of  $u$  runs the for loop over  $\text{adj}[u]$  exactly once, which is  $\deg(u)$  steps. Summing over every vertex that is ever popped, and then over every vertex in  $V$  since that only adds terms:

$$\text{sum over popped } u \text{ of } \deg(u) \leq \text{sum over } u \text{ in } V \text{ of } \deg(u) = 2m$$

That last equality is the counting result proved earlier. Each edge is examined at most twice in total, once from each endpoint, and never once per iteration of an outer loop.

## Adding it up.

At most  $n$  pops with constant work each, at most  $2m$  neighbor steps, and  $\Theta(n)$  to allocate the visited structure over all vertices. Total  $\Theta(n + m)$ .

### WHY THE $N$ TERM SURVIVES

A graph can have many isolated vertices, so  $m$  can be far smaller than  $n$ .  $\Theta(m)$  alone would be wrong, and  $\Theta(n)$  alone would be wrong on a dense graph. Both terms are load bearing.

### THE SAME PROOF WITH A MATRIX

Replace the adjacency list with a matrix and one pop takes  $\Theta(n)$  to scan a row instead of  $\Theta(\deg(u))$ . The total becomes  $\Theta(n^2)$  no matter how few edges exist. The representation, not the algorithm, sets the bound.

### DIRECTED GRAPHS ARE UNCHANGED

The scan of  $\text{adj}[u]$  now walks out neighbors, so the sum is over  $\text{outdeg}(u)$  rather than  $\deg(u)$ , and that sums to  $m$  instead of  $2m$ . Same  $\Theta(n + m)$ .

# ONE COMPONENT AT A TIME

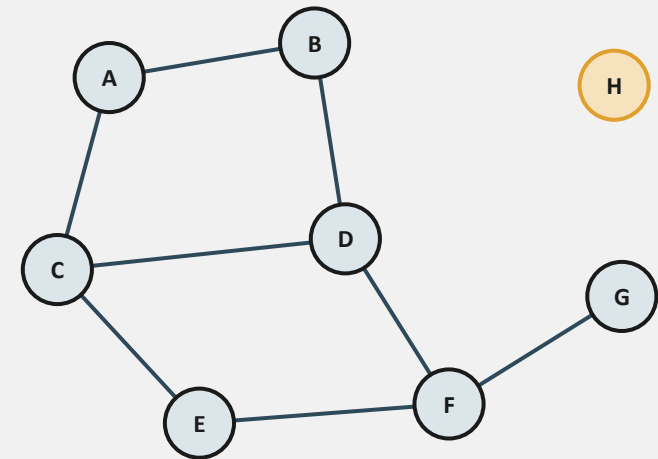
A search from one source reaches exactly what that source can reach. H is still sitting there unvisited, and pretending otherwise is a common bug.

```
def components(adj):  
    seen = set()  
    result = []  
  
    for u in adj:          # every vertex, any order  
        if u not in seen:  
            comp = bfs(adj, u) # exactly u's component  
            seen |= comp  
            result.append(comp)  
  
    return result
```

**On the running graph this returns two sets.**

`[{A, B, C, D, E, F, G}, {H}]`

The outer loop is what makes the traversal total. Without it you have answered a question about one source, which is often not the question that was asked.



## STILL $\Theta(N + M)$ IN TOTAL

The outer loop runs  $n$  times, but the inner searches partition the vertices and the edges among themselves. No vertex and no edge is scanned by two different searches, so the total is  $n + m$  and not  $n$  times  $(n + m)$ .

## THE DIRECTED CASE IS..

On a directed graph this finds what is reachable from  $u$ , which is not the same as a strongly connected component.

# ADDING LAYERS

Replace the visited set with a dictionary of distances. The set was already the right shape, it just was not carrying a number.

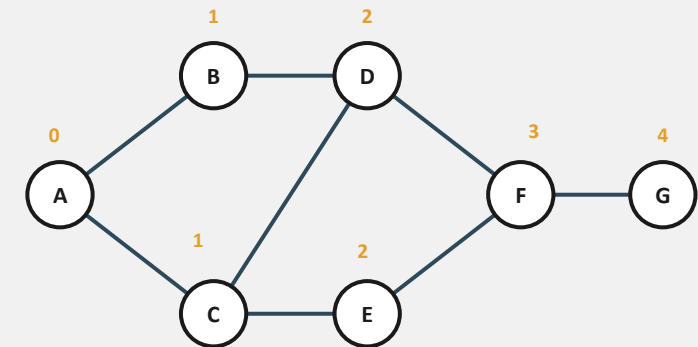
```
def bfs_layers(adj, source):
    layer = {source: 0}          # doubles as visited
    queue = deque([source])

    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in layer:
                layer[v] = layer[u] + 1
                queue.append(v)

    return layer
```

**Three things changed and none of them touched the traversal.**

The set became a dictionary. Membership testing became a key lookup, which is still expected constant time. Marking became an assignment of the parent layer plus one. The running time argument from two slides ago applies word for word, so this is still  $\Theta(n + m)$ .



`layer = {A:0, B:1, C:1, D:2, E:2, F:3, G:4}`

## BACKWARDS WALKING?

Write `parent[v] = u` in the same branch and you can follow the pointers from `v` back to the source afterwards. The layer number is the length of that walk.

## WHEN YOU NEED A WHOLE LAYER AT ONCE

Read `len(queue)` before the round, pop exactly that many vertices, then increment a counter. Same traversal, different bookkeeping.

# THE LAYER IS THE DISTANCE

Write  $d(s, v)$  for the least number of edges on any walk from  $s$  to  $v$ . The claim is that  $\text{layer}[v]$  equals  $d(s, v)$  for every reachable  $v$ .

**Invariant.** At every moment the queue holds vertices of at most two distinct layer values, some  $i$  and  $i + 1$ , with every  $i$  ahead of every  $i + 1$ .

A label is only ever assigned as  $\text{layer}[u] + 1$  where  $u$  is the vertex just popped from the front, and the queue is first in first out. So values are appended in nondecreasing order and vertices are popped in nondecreasing layer order.

**Claim.**  $\text{layer}[v] = d(s, v)$  for every  $v$  reachable from  $s$ .

Induction on  $k = d(s, v)$ .

**Base,  $k = 0$ .**

The only vertex at distance 0 is  $s$  itself, and  $\text{layer}[s] = 0$  by construction.

**Step.**

Assume the claim holds for every vertex at distance  $k$ , and let  $d(s, v) = k + 1$ . Some shortest walk to  $v$  ends with an edge from a vertex  $u$  with  $d(s, u) = k$ , so  $\text{layer}[u] = k$  by hypothesis.

Upper bound. When  $u$  is popped,  $v$  is either already labelled or gets labelled  $k + 1$  right then. If it was labelled earlier it was labelled from some  $w$  popped before  $u$ , and by the invariant  $\text{layer}[w] \leq k$ , so  $\text{layer}[v] \leq k + 1$  either way.

Lower bound. The chain of assignments that produced  $\text{layer}[v]$  traces an actual walk from  $s$  to  $v$  with exactly  $\text{layer}[v]$  edges. No walk to  $v$  is shorter than  $d(s, v)$ , so  $\text{layer}[v] \geq k + 1$ .

**Both bounds give  $\text{layer}[v] = k + 1 = d(s, v)$ .**

## WHY UNWEIGHTED IS DOING THE WORK

Every step of this argument counts edges, and it counts each edge as one. Give the edges different costs and the walk with the fewest edges stops being the walk of least total cost, and the induction breaks at the upper bound.

(Dijkstra's fixes this)

## SHORTEST WALK IS HERE

Single source shortest walks in an unweighted graph, directed or undirected, in  $\Theta(n + m)$ . No extra data structure, no extra pass, and the same twelve lines of code.